

SUPSI

Branch and Bound as PTAS

Studente/i

Tiziano Zamaroni

Sergio Aquilini

Relatore

Monaldo Mastrolilli

Correlatore

-

Committente

SUPSI

Corso di laurea

C-P6051P - Seminari di Progetto

Modulo

**M-P6050P - Progetto di
Semestre**

Anno

2025

Data

14 dicembre 2025

Indice

1	Introduzione	1
2	Motivazione e Contesto	3
2.1	Motivazione	3
2.2	Contesto	3
3	Algoritmi	5
3.1	Approssimazione soluzione	5
3.2	Branch-and-Bound	5
3.3	Greedy	5
3.4	Profiling	5
4	Requisiti	7
4.1	Requisiti funzionali	7
4.2	Requisiti non funzionali	8
5	Tecnologie utilizzate	9
5.1	Implementazione algoritmi	9
5.2	Analisi dei dati	9
6	Implementation	11
6.1	Architettura e struttura del codice	11
6.2	Branch-and-Bound	11
6.2.1	Implementation Overview	11
6.2.2	Pseudocode	14
6.2.3	Lower-Bound Strategies	15
6.2.4	Rounding Strategies	16
6.2.5	Branching Rules	17
6.2.6	Node Selection Strategies	18
6.3	Adaptation to Identical Machines Scheduling	18
6.3.1	Key Differences with Respect to the Unrelated Case	19

6.3.2	Bounding for Identical Machines	19
6.3.2.1	LP-based approach and rounding incompatibility	19
6.3.2.2	Greedy fractional bound	20
6.3.3	Rounding Strategies for Identical Machines	20
6.3.4	Stopping Criterion and Approximation Guarantee	21
6.3.5	Profiling-Based Prioritisation and Pruning	21
6.3.5.1	Profiling modes	21
6.3.5.2	Normalization and big jobs	22
6.3.5.3	Geometric binning	22
6.3.5.4	Construction of profile keys	22
6.3.5.5	Profile comparison and depth-based equivalence	23
6.4	Flexible Instance Generation and Experimental Configuration	23
6.5	Analysis and Data Exploration Pipeline	24
7	Analisi dei risultati	27
8	Metodologia di lavoro	29
9	Conclusione	31
9.1	Risultati ottenuti	31
9.2	Possibili miglioramenti futuri	31

Elenco delle figure

Elenco delle tabelle

1 Introduzione

- Problemi NP-Hard
- Branch-and-Bound Algorithms as Polynomial-time Approximation Schemes
- Unrelated parallel machine scheduling problem
- Uniform/Identical parallel machine scheduling problem (identical caso particolare di uniform)
- Profiling come ulteriore ottimizzazione di identical machine scheduling
- Dimostrazione pratica della validità dell'approccio con profiling

2 Motivazione e Contesto

2.1 Motivazione

2.2 Contesto

3 Algoritmi

3.1 Approssimazione soluzione

Differenze del valore di epsilon tra i vari problemi

3.2 Branch-and-Bound

3.3 Greedy

3.4 Profiling

4 Requisiti

Il lavoro di semestre si inserisce nel contesto del paper *Branch-and-Bound Algorithms as Polynomial-time Approximation Scheme* [1], scritto dal nostro relatore Prof. Monaldo Mastroilli insieme a István Encz Koppányi ed Eleonora Vercesi. Gli autori forniscono un'implementazione Python del Branch-and-Bound per il caso delle *unrelated machines*, comprensiva di euristiche e gestione dei bound. Il nostro compito consiste nel comprendere a fondo tale implementazione, adattarla al caso delle *identical machines* e integrare la tecnica di *profiling*, applicabile poiché le identical machines rappresentano un caso particolare di uniform machines. L'obiettivo finale è verificare sperimentalmente quanto presentato nel paper, confrontando la versione base dell'algoritmo con quella dotata di profiling.

4.1 Requisiti funzionali

Il progetto richiede i seguenti requisiti funzionali:

RF1 – Solver per identical machines

Realizzare una variante dell'algoritmo Branch-and-Bound in grado di risolvere il problema di scheduling su identical machines, adattando la struttura e le funzioni dell'implementazione esistente per le unrelated machines.

RF2 – Integrazione del profiling

Implementare il meccanismo di profiling descritto nel paper, così da ottenere una seconda variante dell'algoritmo direttamente confrontabile con la versione base.

RF3 – Generazione ed esecuzione degli esperimenti

Eseguire entrambe le varianti dell'algoritmo e registrare i risultati, raccogliendo una quantità consona di dati.

RF4 – Raccolta dei dati

Memorizzare in formato strutturato informazioni quali nodi esplorati, tempo di esecuzione, delta rispetto alla soluzione ottima dell'istanza considerata e altre metriche rilevanti.

RF5 – Validazione sperimentale

Analizzare i dati raccolti, con l'aiuto di strumenti statistici e di visualizzazione, per verificare l'efficacia del profiling rispetto alla versione base dell'algoritmo.

4.2 Requisiti non funzionali

Non erano previsti requisiti non funzionali formali, ma il progetto implicava alcuni vincoli pratici:

Efficienza computazionale

L'implementazione e la scelta delle istanze dovevano permettere di generare e analizzare un numero significativo di dati entro i limiti temporali del lavoro di semestre.

Ripetibilità

La generazione delle istanze e l'esecuzione degli algoritmi dovevano essere riproducibili tramite seed controllati.

Mantenibilità e pulizia del codice

Anche trattandosi di un esperimento non destinato a evoluzione futura, il codice doveva essere scritto in modo chiaro e integrarsi coerentemente con la struttura esistente.

5 Tecnologie utilizzate

5.1 Implementazione algoritmi

L'implementazione degli algoritmi è stata sviluppata in *Python* a partire dal codice fornito dagli autori del paper [1], originariamente progettato per il caso delle *unrelated machines*. In tale versione, il lower bound dei nodi del Branch-and-Bound veniva calcolato tramite il solver *SCIP* [2], utilizzato attraverso la libreria *PySCIPOpt*. Questa scelta, appropriata per il problema generale, consente agli autori di ottenere bound frazionari mediante *binary search* e *linear relaxation*.

Nel nostro progetto, tuttavia, tale approccio si è rivelato eccessivamente oneroso e non strettamente necessario. Nel caso delle *identical machines*, infatti, la struttura del problema è significativamente più semplice e permette l'impiego di metodi più leggeri per il calcolo del lower bound. Inoltre, l'utilizzo di *SCIP* [2] presentava un ulteriore limite pratico: non è stato possibile forzare il solver a utilizzare sistematicamente il metodo *primal simplex*, condizione necessaria per garantire che il numero di variabili frazionarie nella soluzione LP rimanesse inferiore o uguale al numero di macchine. L'assenza di tale garanzia comprometteva l'affidabilità della fase di arrotondamento e introduceva un overhead computazionale non trascurabile.

Per queste ragioni, abbiamo sostituito integralmente l'utilizzo di *SCIP* [2] con un metodo di calcolo del lower bound basato su un criterio *greedy*, pienamente sufficiente per il caso delle *identical machines* e decisamente più efficiente dal punto di vista computazionale. Questa scelta ha permesso di ridurre drasticamente il tempo di elaborazione dei nodi del Branch-and-Bound, mantenendo comunque una qualità del bound adeguata agli scopi del progetto.

5.2 Analisi dei dati

L'analisi dei dati sperimentali è stata effettuata in *Python* utilizzando le principali librerie per la data science. *numpy* è stato impiegato per l'elaborazione numerica di base, *pandas* per l'organizzazione e l'aggregazione dei risultati raccolti durante gli esperimenti, e *matplotlib* per la generazione delle visualizzazioni utilizzate nel confronto tra la versione base

dell'algoritmo e quella dotata di profiling. I dati sono stati esportati in formato CSV e analizzati tramite notebook dedicati, permettendo di sintetizzare l'andamento delle performance e supportare quantitativamente la discussione dei risultati.

6 Implementation

6.1 Architettura e struttura del codice

Still needed??

6.2 Branch-and-Bound

The implementation employed in this project is based on a branch-and-bound (B&B) framework originally developed for the *unrelated machines scheduling* problem. The same structure is later reused for the identical machines case, where only the lower-bound routines and selected optimisation components (such as profiling) are adapted, while the core branching, bounding, rounding, and node-selection logic is preserved.

6.2.1 Implementation Overview

Each node of the branch-and-bound tree represents a subproblem corresponding to a partial assignment of jobs to machines and is characterised by:

- a set of fixed assignments $\text{fixed} \subseteq \{0, \dots, n-1\} \times \{0, \dots, m-1\}$,
- an overhead vector $h \in \mathbb{R}^m$, where h_i denotes the load already assigned to machine i ,
- a fractional solution X^{frac} obtained from a relaxation of the problem,
- a node lower bound LB and a node upper bound UB,
- the depth of the node in the search tree.

For a given node, the lower bound LB denotes a valid lower bound on the optimal makespan of the corresponding subproblem, obtained from a relaxation, while the upper bound UB corresponds to the makespan of a feasible integer schedule associated with that node.

The global state maintains a best-known feasible solution, represented by a global upper bound LUB, and a global lower bound LLB derived from the set of active nodes in the search tree. More precisely, the global upper bound LUB corresponds to the makespan of the best

feasible (integral) schedule found so far, while the global lower bound LLB is defined as the minimum lower bound among all active nodes in the priority queue.

The behaviour of the algorithm is governed by four main components.

Bounding. For each node, a lower bound on the optimal makespan of the associated subproblem is computed by solving either a linear relaxation or a feasibility-based binary search (see Section 6.2.3). Nodes whose lower bound is not strictly smaller than the current global upper bound LUB are pruned.

Branching. When the relaxation at a node yields a non-integer solution, one fractional job is selected according to a branching rule (see Section 6.2.5). This job is fixed on each possible machine, generating up to m child nodes and partitioning the remaining search space.

Rounding. A feasible integer solution is obtained from the fractional relaxation through a deterministic rounding strategy (see Section 6.2.4). The resulting makespan defines a node upper bound UB, which may improve the current global best solution.

Node selection. Active nodes are stored in a priority queue and extracted according to a configurable selection strategy (see Section 6.2.6). The choice of strategy affects the traversal order of the search tree but not the correctness of the algorithm.

The algorithm terminates when the relative optimality gap satisfies

$$\frac{\text{LUB}}{\text{LLB}} < 1 + \varepsilon,$$

indicating that the incumbent solution is within a factor $(1 + \varepsilon)$ of optimality, or when the search space has been fully explored. In practice, an additional limit on the maximum number of explored nodes may be enforced to prevent excessive runtimes.

6.2.2 Pseudocode

Algorithm 1 Branch-and-bound framework (unrelated machines scheduling)

Require: instance $p_{j,i}$, tolerance ε ; strategies for node selection, bounding, branching, rounding

```

1:  $(X^{\text{frac}}, \text{LB}) \leftarrow \text{LowerBound}(p, h=0, \text{fixed}=\emptyset)$ 
2: if  $X^{\text{frac}}$  is integral then return  $(X^{\text{frac}}, \text{LB})$ 
3: end if
4:  $(X^*, \text{LUB}) \leftarrow \text{Rounding}(X^{\text{frac}}, \emptyset)$ 
5: Initialize queue  $Q$  with root node ( $\text{fixed}=\emptyset, h=0, X^{\text{frac}}, \text{LB}$ )
6:  $\text{LLB} \leftarrow \text{LB}$ 
7: while  $Q \neq \emptyset$  do
8:    $\text{LLB} \leftarrow \min(\text{LUB}, \min_{N \in Q} N.\text{LB})$ 
9:   if  $\text{LUB}/\text{LLB} < 1 + \varepsilon$  then break
10:  end if
11:  Extract node  $N$  from  $Q$  according to the node selection strategy
12:  Select a fractional job  $j$  from  $N.X^{\text{frac}}$  (branching rule)
13:  for  $q \in \{0, \dots, m-1\}$  do ▷ create child by fixing  $(j, q)$ 
14:     $\text{fixed}' \leftarrow N.\text{fixed} \cup \{(j, q)\}, \quad h' \leftarrow N.h \text{ with } h'_q \leftarrow h_q + p_{j,q}$ 
15:    if  $\text{fixed}'$  assigns all jobs then
16:       $\text{UB}' \leftarrow \max_i h'_i$ 
17:      if  $\text{UB}' < \text{LUB}$  then  $(X^*, \text{LUB}) \leftarrow (\text{fixed}', \text{UB}')$ 
18:      end if
19:      continue
20:    end if
21:     $(X^{\text{frac}'}, \text{LB}') \leftarrow \text{LowerBound}(p, h', \text{fixed}')$ 
22:    if  $\text{LB}' \geq \text{LUB}$  then continue ▷ prune by bound
23:    end if
24:    if  $X^{\text{frac}'}$  is integral then
25:      if  $\text{LB}' < \text{LUB}$  then  $(X^*, \text{LUB}) \leftarrow (X^{\text{frac}'}, \text{LB}')$ 
26:      end if
27:      continue
28:    end if
29:     $(X^{\text{int}'}, \text{UB}') \leftarrow \text{Rounding}(X^{\text{frac}'}, \text{fixed}')$ 
30:    if  $\text{UB}' < \text{LUB}$  then  $(X^*, \text{LUB}) \leftarrow (X^{\text{int}'}, \text{UB}')$ 
31:    end if
32:    Insert child node  $(\text{fixed}', h', X^{\text{frac}'}, \text{LB}')$  into  $Q$ 
33:  end for
34: end while
35: return  $(X^*, \text{LUB})$ 

```

Algorithm 1 summarises the inherited branch-and-bound framework used for the unrelated machines scheduling problem. For each node, a relaxation is solved to obtain a fractional solution X^{frac} and a lower bound LB on the optimal makespan of the corresponding subproblem.

A global upper bound LUB is maintained through feasible integer solutions produced either by rounding fractional solutions (Section 6.2.4) or by integrality of the relaxation. Whenever a solution with smaller makespan is found, the incumbent solution X^* and the global upper bound are updated.

The search space is explored by repeatedly extracting nodes from a priority queue according to the selected node selection strategy. If the extracted node is fractional, a branching rule selects one fractional job j ; branching fixes j on each machine, generating up to m child nodes. For each child node, a new lower bound LB' is computed and the node is immediately discarded if $LB' \geq LUB$, since it cannot lead to an improvement over the incumbent solution. The global lower bound LLB is derived from the minimum lower bound among the nodes currently stored in the queue, clipped by LUB.

6.2.3 Lower-Bound Strategies

The inherited implementation provides two alternative strategies to compute node-level lower bounds within the branch-and-bound framework. Both operate on the subproblem defined by:

- the set of fixed assignments *fixed*,
- the overhead vector $h \in \mathbb{R}^m$,
- the remaining unfixed jobs.

In both cases, the output consists of a fractional assignment X^{frac} and a scalar lower bound LB.

Linear relaxation (lin_relax). In the linear-relaxation strategy, a linear program is solved in which each unfixed job may be fractionally assigned to machines. Let $x_{j,i} \geq 0$ denote the fraction of job j assigned to machine i , and let $C_{\max}$ denote the makespan. The model can be expressed as:

$$\begin{aligned}
 & \min C_{\max} \\
 & \text{s.t. } \sum_{i=0}^{m-1} x_{j,i} = 1 && \forall j \text{ unfixed,} \\
 & \sum_{j \text{ unfixed}} p_{j,i} x_{j,i} \leq C_{\max} - h_i && \forall i = 0, \dots, m-1, \\
 & x_{j,i} \geq 0 && \forall j, i.
 \end{aligned}$$

The fixed jobs do not appear as decision variables; their contribution is encoded in the overhead h . The optimal value $C_{\max}^*$ is used as the node lower bound LB, and the corresponding solution defines X^{frac} . Due to the structure of the formulation, at most m jobs are fractional in the optimal solution.

Binary search on the makespan (bin_search). The second strategy performs a binary search on the makespan value. An interval $(\ell, r]$ is maintained such that ℓ is infeasible and r is feasible. For a candidate makespan C , a feasibility LP is solved with variables $x_{j,i}$ for unfixed jobs, subject to:

$$\begin{aligned}
 \sum_{i=0}^{m-1} x_{j,i} &= 1 && \forall j \text{ unfixed,} \\
 \sum_{j \text{ unfixed}} p_{j,i} x_{j,i} &\leq C - h_i && \forall i = 0, \dots, m-1, \\
 x_{j,i} &= 0 && \text{if } p_{j,i} > C - h_i, \\
 x_{j,i} &\geq 0 && \text{otherwise.}
 \end{aligned}$$

If the LP is feasible, the value C becomes the new right endpoint; otherwise it becomes the new left endpoint. The process continues until $(\ell, r]$ cannot be further refined, and the final feasible value r is taken as the node lower bound. The corresponding LP solution defines the fractional assignment X^{frac} .

6.2.4 Rounding Strategies

When the relaxation at a node yields a fractional solution X^{frac} , a deterministic rounding procedure is applied to construct a feasible integer assignment. The resulting schedule defines a node upper bound UB, which may improve the current global upper bound LUB.

The rounding implementation operates on the subproblem defined by the fixed assignments fixed and the overhead vector $h \in \mathbb{R}^m$. Jobs already fixed do not appear as decision variables; their contribution is accounted for by the overhead values h_i . Moreover, jobs that are already integrally assigned in X^{frac} are preserved.

Let F denote the set of *fractional jobs*, i.e., those jobs j such that $x_{j,i}$ is non-integer for at least one machine i . The implementation assumes $|F| \leq m$, which is consistent with the structure of the relaxations used in Section 6.2.3. Rounding proceeds by first computing the machine completion times induced by (i) integrally assigned jobs in X^{frac} and (ii) fixed jobs, and then assigning the jobs in F according to one of the following strategies.

Assign-to-shortest machine (`all_to_shortest`). Each fractional job $j \in F$ is assigned to the machine that minimises its processing time:

$$i^*(j) \in \arg \min_{i=0, \dots, m-1} p_{j,i}.$$

The completion times are updated accordingly, and the upper bound is obtained as the makespan of the resulting schedule:

$$\text{UB} = \max_{i=0, \dots, m-1} C_i,$$

where C_i denotes the completion time of machine i . This strategy is computationally inexpensive and purely greedy with respect to individual job processing times.

Best matching of fractional jobs (`best_matching`). Since $|F| \leq m$, the fractional jobs can be assigned by enumerating all injective mappings of F to machines. Equivalently, all permutations of machines of length $|F|$ are considered. For each candidate placement, the completion times are updated and the makespan is evaluated; the assignment yielding the minimum makespan among the enumerated placements is selected, and the corresponding makespan is returned as UB.

6.2.5 Branching Rules

When the relaxation at a node produces a non-integer solution, a set of *fractional jobs* is obtained, namely those jobs j for which at least one variable $x_{j,i}$ is non-integer. The implementation assumes that the number of such jobs is at most the number of machines m , which is guaranteed by the structure of the relaxations. A branching rule selects one job j from this set, and the node is branched by fixing job j on each machine $q = 0, \dots, m-1$, generating up to m children.

Two selection criteria are provided.

Maximum minimum processing time (`max_min_proc`). For each fractional job j , the quantity

$$\min_{i=0, \dots, m-1} p_{j,i}$$

is considered. The branching job is the one that maximises this minimum processing time. This rule gives priority to jobs that are relatively expensive even on their best machine, so that branching decisions focus on jobs with high potential impact on the makespan.

Maximum average processing time (max_avg_proc). For each fractional job j , the average processing time

$$\frac{1}{m} \sum_{i=0}^{m-1} p_{j,i}$$

is computed. The branching job is the one with maximum average processing time, i.e., the job that is globally heaviest across all machines. This tends to branch first on jobs that dominate the total processing workload.

6.2.6 Node Selection Strategies

The branch-and-bound framework explores the search space by maintaining a set of active nodes in a priority queue. The order in which nodes are extracted from this queue defines the exploration strategy. While node selection does not affect the correctness of the algorithm, it can significantly influence the order in which feasible solutions are found and, consequently, the effectiveness of pruning.

Each node is associated with a priority value derived from its attributes, and nodes are compared according to a configurable selection strategy. The implementation supports the following strategies.

Lowest lower bound first. Nodes are ordered by increasing node lower bound LB. At each iteration, the node with the smallest lower bound is selected. This strategy prioritises subproblems that appear most promising with respect to the objective and is commonly used in best-first branch-and-bound variants.

Depth-first search. Nodes with larger depth in the search tree are prioritised. This strategy explores complete assignments early, which may quickly produce good feasible solutions and improve the global upper bound, potentially increasing pruning opportunities.

Breadth-first search. Nodes are selected in order of increasing depth. This strategy explores the search tree level by level and provides a more uniform coverage of the solution space, at the cost of potentially delaying the discovery of good feasible solutions.

6.3 Adaptation to Identical Machines Scheduling

This section describes the adaptation of the inherited branch-and-bound (B&B) framework to the *identical machines scheduling* problem, which constitutes the main scope of this project. While the overall structure of the algorithm remains unchanged, the identical-machines setting enables problem-specific simplifications and optimisations that significantly affect the bounding procedures, rounding strategies, stopping criterion, and the integration of profiling mechanisms inspired by the PTAS described in the reference paper.

All theoretical ideas originate from the paper; the contribution of this project lies in their faithful translation into a robust and efficient implementation, addressing practical issues that arise when embedding PTAS concepts into a branch-and-bound framework.

6.3.1 Key Differences with Respect to the Unrelated Case

In the identical-machines setting, processing times depend only on the job index, i.e. $p_{j,i} = p_j$ for all machines i . This structural simplification leads to the following consequences:

- The fractional optimum for the root relaxation admits a closed form, given by the average load.
- Lower-bound computations can be simplified and made solver-independent.
- Rounding strategies can exploit the symmetry between machines and rely on current machine loads.
- Profiling-based prioritisation and pruning can be integrated to reduce the effective search space.
- The stopping criterion must be relaxed to account for the additional approximation introduced by profiling.

The following subsections describe these adaptations in detail.

6.3.2 Bounding for Identical Machines

6.3.2.1 LP-based approach and rounding incompatibility

An initial attempt to reuse the LP-based bounding strategies employed for the unrelated-machines case relied on solving linear relaxations via a generic LP solver. However, the rounding procedures used throughout the framework rely on the structural property that the number of fractional jobs in an optimal relaxation solution is bounded by the number of machines m .

In practice, the LP solver occasionally returned fractional solutions that violated this assumption, for instance by distributing a large number of jobs fractionally across machines. While such solutions are theoretically valid, they are incompatible with the in-place rounding procedures assumed by the framework, causing rounding to become ill-defined or inefficient. Since enforcing a specific extreme-point structure on the solver output proved impractical, a different approach was adopted for the identical-machines case.

6.3.2.2 Greedy fractional bound

For identical machines, the relaxation admits a much simpler structure. Let

$$C_{\max}^{\text{frac}} = \frac{1}{m} \sum_{j=1}^n p_j$$

denote the fractional optimum makespan in the absence of fixed jobs. The greedy bound constructs a feasible fractional assignment that respects the current fixed assignments and machine loads, and fills the remaining capacity up to $C_{\max}^{\text{frac}}$.

The procedure operates as follows:

- The current machine loads induced by fixed jobs are stored in an overhead vector h .
- Unfixed jobs are processed sequentially.
- Each job is fractionally assigned to machines whose current load is below $C_{\max}^{\text{frac}}$, until its processing time is fully distributed.

The resulting fractional assignment is feasible by construction and yields a valid lower bound for the corresponding subproblem. Moreover, the greedy procedure is deterministic and produces fractional solutions compatible with the rounding strategies used in the framework.

Algorithm 2 Greedy fractional bound for identical machines

Require: processing times p_j , overhead h , number of machines m

```

1:  $C_{\max} \leftarrow \frac{1}{m} \sum_j p_j$ 
2:  $C_i \leftarrow h_i$  for all machines  $i$ 
3: for each unfixed job  $j$  do
4:    $r \leftarrow p_j$ 
5:   while  $r > 0$  do
6:     select a machine  $i$  with  $C_i < C_{\max}$ 
7:      $\delta \leftarrow \min(r, C_{\max} - C_i)$ 
8:     assign fraction  $\delta/p_j$  of job  $j$  to machine  $i$ 
9:      $C_i \leftarrow C_i + \delta$ ,  $r \leftarrow r - \delta$ 
10:  end while
11: end for
12: return fractional assignment and makespan  $\max_i C_i$ 
```

6.3.3 Rounding Strategies for Identical Machines

Given a fractional solution X^{frac} , a deterministic rounding procedure is applied to obtain a feasible integer schedule. Jobs that are already integrally assigned in X^{frac} , as well as fixed jobs, are preserved. Let F denote the set of fractional jobs.

Assign-to-shortest machine (`all_to_shortest`). Each fractional job $j \in F$ is assigned to the machine with the smallest current completion time. This greedy strategy exploits the symmetry of identical machines and incrementally balances the load. The resulting makespan defines the node upper bound UB.

First-machine assignment (`first_machine`). Each fractional job is assigned to the first machine it appears on in the fractional solution. This rule provides a simple, deterministic baseline and is primarily used for testing and comparison.

Best matching (`best_matching`). When $|F| \leq m$, all possible assignments of the fractional jobs to machines are enumerated and the one minimising the resulting makespan is selected. Although exponential in $|F|$, this strategy remains practical due to the small number of fractional jobs.

6.3.4 Stopping Criterion and Approximation Guarantee

In the absence of profiling, the stopping criterion coincides with the standard relative gap condition:

$$\frac{\text{LUB}}{\text{LLB}} < 1 + \varepsilon,$$

which guarantees a $(1 + \varepsilon)$ -approximate solution.

When profiling is enabled, additional pruning is performed based on profile equivalence. As prescribed by the reference paper, this introduces an extra approximation factor, which must be accounted for in the termination condition. Accordingly, the algorithm is allowed to terminate when

$$\frac{\text{LUB}}{\text{LLB}} < (1 + \varepsilon)^2.$$

For the same input value of ε , this condition admits a larger relative optimality gap than in the non-profiling case. The relaxed stopping criterion compensates for the approximation introduced by profile-based pruning and ensures that the final solution satisfies the approximation bound established in the paper.

6.3.5 Profiling-Based Prioritisation and Pruning

Profiling is introduced to reduce the effective search space by identifying nodes that induce similar load distributions. Each node is associated with a *profile key* that abstracts the distribution of machine loads induced by large jobs.

6.3.5.1 Profiling modes

Three profiling modes are supported:

- `NO_PROFILING`: profiling is disabled.

- **PRIORITY:** nodes without a previously seen profile are prioritised in the queue.
- **PRUNE:** nodes whose profile matches one already seen at the same depth are discarded.

6.3.5.2 Normalization and big jobs

Let

$$T_{\text{norm}} = \frac{1}{m} \sum_j p_j$$

denote the normalization scale. A job j is classified as *big* if its processing time is significant with respect to the normalization scale, namely if

$$p_j \geq \varepsilon \cdot T_{\text{norm}}.$$

This threshold follows the PTAS framework for identical machines, where jobs smaller than $\varepsilon \cdot T_{\text{norm}}$ are treated as negligible at the chosen approximation scale. Accordingly, only the contribution of big jobs is considered when computing profiling information.

For implementation efficiency, each node explicitly stores, in addition to the total machine loads h , a separate vector $h^{\text{big}} \in \mathbb{R}^m$ containing only the load contribution of big jobs. This vector is incrementally updated during branching and is used exclusively for profiling, so that profile computation does not require recomputing or filtering job assignments.

6.3.5.3 Geometric binning

Machine loads are normalized by T_{norm} and discretized using a geometric grid:

$$0, \varepsilon, \varepsilon(1 + \varepsilon), \varepsilon(1 + \varepsilon)^2, \dots, 2(1 + \varepsilon)^2.$$

This discretization preserves relative load differences while tolerating small perturbations, as required by the PTAS framework.

6.3.5.4 Construction of profile keys

For each node, profiling is based on the distribution of machine loads induced by big jobs only. Let $h^{\text{big}} \in \mathbb{R}^m$ denote the vector of machine loads obtained by summing only the processing times of big jobs assigned to each machine.

Each component is normalized as

$$\tilde{h}_i = \frac{h_i^{\text{big}}}{T_{\text{norm}}}.$$

The normalized loads are then mapped to the geometric bins, and a histogram over all machines is constructed. This histogram constitutes the *profile key* of the node.

6.3.5.5 Profile comparison and depth-based equivalence

Two nodes are considered ε -equivalent if they:

- occur at the same depth in the branch-and-bound tree, and
- share the same profile key.

The implementation maintains, for each depth, a set of profile keys that have already been observed. When a new node is generated, its profile key is computed and compared against this set. If an identical key is found, the node is marked as having a similar profile.

Algorithm 3 Profiling and equivalence detection

Require: node with big-job overhead h^{big} at depth d

- 1: normalize loads: $\tilde{h}_i \leftarrow h_i^{\text{big}} / T_{\text{norm}}$
 - 2: discretize $\tilde{h}_i$ using geometric bins
 - 3: construct histogram K
 - 4: **if** K already seen at depth d **then**
 - 5: mark node as having similar profile
 - 6: **else**
 - 7: store K as seen at depth d
 - 8: **end if**
-

Depending on the selected profiling mode, this information is used either to deprioritize the node in the priority queue or to discard it entirely before insertion.

6.4 Flexible Instance Generation and Experimental Configuration

The inherited experimental setup allowed testing multiple problem sizes by specifying pairs $(n_{\text{jobs}}, n_{\text{machines}})$ and iterating over a fixed range of random seeds. While sufficient for basic validation, this approach provided limited flexibility when exploring the behaviour of the algorithm under different parameter combinations.

To support a more systematic and exploratory experimental analysis, the instance-generation pipeline was extended to decouple *instance specification* from *algorithmic configuration*. In the revised setup, an experiment is defined by explicitly pairing a concrete problem instance with a specific set of algorithmic parameters.

More precisely, each experiment is characterized by:

- the number of machines and jobs,
- the distribution and range of processing times,

- the random seed used to generate a concrete realization,
- and the algorithmic configuration under which the instance is evaluated (including profiling mode, bounding strategy, branching rule, rounding rule, node selection strategy, and tolerance ε).

This separation enables fine-grained control over the experimental space. The same instance can be evaluated under multiple algorithmic variants, and conversely, specific configurations can be restricted to carefully selected families of instances. This flexibility proved essential during the exploratory phase of the project, allowing the identification of meaningful parameter ranges and combinations worthy of deeper analysis.

In addition, to reduce computational overhead and enable large-scale testing, exact optimal solutions were no longer computed for every instance. Instead, the fractional optimum for identical machines, given by $\sum_j p_j / m$, was used as a consistent baseline for performance evaluation. This choice is sufficient for measuring approximation quality and aligns with the theoretical framework of the PTAS, while significantly improving experimental throughput.

Finally, this setup made it possible to directly compare configurations with and without profiling under comparable approximation guarantees. Since profiling introduces a more relaxed stopping condition, non-profiling configurations were evaluated using a correspondingly adjusted tolerance, ensuring that all comparisons refer to equivalent approximation levels.

6.5 Analysis and Data Exploration Pipeline

All experimental runs were automatically logged into structured CSV files, with each row corresponding to a single execution of the branch-and-bound algorithm under a specific instance–configuration pair. The recorded data include instance characteristics, algorithmic parameters, runtime statistics, search metrics (e.g. explored nodes, depth), and solution quality indicators.

To analyse and interpret these results, a set of Python notebooks was developed. These notebooks served as an exploratory and analytical layer on top of the raw experimental data, supporting data filtering, aggregation, visualization, and comparison across configurations. Standard data-analysis libraries were used to load the CSV files, manipulate the datasets, and generate plots illustrating trends and performance trade-offs.

The notebooks were primarily used for two purposes. First, they enabled an exploratory analysis phase, during which significant ranges of parameters (e.g. tolerance ε , instance sizes, and profiling modes) were identified. This phase guided the selection of meaningful experimental configurations and helped exclude combinations that were either uninformative or computationally impractical.

Second, once the experimental scope was consolidated, the same analysis pipeline was used to produce the plots and summary statistics presented in Chapter ???. In this phase,

the notebooks acted as a reproducible tool for aggregating results and visualizing the impact of profiling, tolerance levels, and instance scale on both runtime and solution quality.

The separation between raw data collection and analysis ensured that experimental results could be re-examined, filtered, and visualized without rerunning the computationally expensive experiments, and facilitated an iterative workflow between experimentation and analysis.

7 Analisi dei risultati

- Overview istanze e dati raccolti
- Istanze relativamente piccole ma con diversi seed
- Difficoltà variabile a dipendenza del seed, a parità di dimensione
- Statistiche no-profiling vs. prune per tipo di istanza
- Evidenziare che profiling rimane percorribile quando la versione base non arriva a soluzione

8 Metodologia di lavoro

- Suddivisione dei compiti
- Pianificazione e Task
- Pair programming / brainstorming
- Weekly
- Raccolta dei dati

9 Conclusione

9.1 Risultati ottenuti

9.2 Possibili miglioramenti futuri

Elenco di possibili sviluppi futuri:

- abc
- def

Bibliografia

- [1] Koppányi István Encz, Monaldo Mastrolilli, and Eleonora Vercesi. Branch-and-bound algorithms as polynomial-time approximation scheme. *Mathematics of Operations Research*, 2025.
- [2] Scip optimization suite. <https://scipopt.org>.